

Self-Healing AI CI/CD Demo

This project is a small Flask API plus a self-healing AI pipeline dashboard. The split is intentional: `app.py` is the application under test, while `dashboard.py` stays available even when `app.py` is intentionally broken for the demo..

`ci_ai.py` is the agent brain. It reads CI logs, uses OpenAI in real mode, classifies the failure, calculates confidence/risk, and decides whether the pipeline can self-heal or must stop for human approval.

Demo Goal

The demo shows three self-healing patterns:

1. **DEV pipeline:** deterministic failures such as syntax errors trigger the AI agent, a safe demo patch, a repair PR, and auto-merge.
2. **STAGING pipeline:** failing tests trigger the AI agent. If confidence is at least 90% and risk is low, the repair patch is pushed and auto-merged.
3. **MAIN pipeline:** production or deployment failures trigger AI triage only. Humans still approve config changes, rollback, or deployment.

Yes, this can be done inside the pipeline. In GitHub Actions, use `if: failure()` or a follow-up job with `needs` and `if: always()` so a failed validation step triggers AI remediation or triage.

The original failed workflow still fails correctly. AI analysis and repair PR creation happen after the failure so the CI signal is not hidden.

Run Locally

```
pip install -r requirements.txt
pytest test_app.py -v
python app.py
```

The API runs on:

```
http://localhost:5000
```

Run the dashboard separately:

```
python dashboard.py
```

Open the dashboard:

```
http://localhost:5050
```

The dashboard shows the latest GitHub Actions run for DEV, STAGING, and MAIN. If a latest run failed and GitHub credentials are configured, it fetches the run logs and shows the self-healing agent decision directly on the dashboard.

Useful endpoints:

Dashboard on port 5050:

GET	/	Dashboard
GET	/dev	DEV analyzer dashboard
GET	/staging	STAGING analyzer dashboard
GET	/main	MAIN analyzer dashboard
GET	/pipeline	AI CI/CD demo metadata
GET	/api/builds	Latest build status for all stages
GET	/api/builds/dev	Latest DEV build with AI failure analysis
POST	/api/analyze	JSON analyzer endpoint

API under test on port 5000:

GET	/health	Service health
GET	/users	List users
POST	/users	Create user with {"name": "John"}

Real AI and GitHub Fetch Mode

Fallback mode works without secrets. Real AI mode uses OpenAI when this variable is configured:

```
OPENAI_API_KEY=your_key
OPENAI_MODEL=gpt-5-mini
```

The dashboard can fetch the latest failed GitHub Actions run when these variables are set:

```
GITHUB_TOKEN=your_token
GITHUB_REPOSITORY=owner/repo
```

GitHub Actions now shows a separate AI agent job after a failed validation job:

DEV validation	failed
AI analyzer	completed with root cause, risk, and repair decision
AI repair PR	creates and auto-merges the repair PR

For **main**, the flow is intentionally different:

MAIN validation	failed
AI analyzer	completed with triage and human-gate decision
No repair PR	production remains human-gated

The agent job writes the result into the GitHub Actions summary with:

```
python scripts/analyze_ci_failure.py --stage staging --log-file ci-logs/staging.log --mode real_ai
```

The agent imports shared logic from `ci_ai.py`, not from `app.py`. That is important because DEV can break `app.py` with a syntax error while the self-healing agent still runs successfully.

Failure Scripts

1. DEV - Syntax Error

```
cp app_broken_dev.py app.py
python -m py_compile app.py
```

Expected result: compile fails. The dashboard still runs from `dashboard.py`; the pipeline analyzes the failure and can open a repair PR for `app.py`.

2. STAGING - Logic Error

```
cp app_broken_staging.py app.py
pytest test_app.py -v
```

Expected result: tests fail. AI should analyze the assertion failure and suggest the logic fix, then wait for review.

3. MAIN - Config Error

```
cp app_broken_main.py app.py
python app.py
```

Expected result: app fails because `DB_HOST` is missing. AI should summarize the cause and remediation choices, but the production decision remains human-approved.

Only workflow files inside `.github/workflows/` are active. Root-level duplicate YAML files were removed to avoid confusion.

Pipeline Pattern

Use this structure for failure-triggered AI:

```
- name: Run validation
  run: pytest test_app.py -v

- name: AI failure analysis
  if: failure()
  run: |
    echo "Collect logs, test output, and changed files"
    echo "Ask AI to produce a fix, PR, or triage report"
```

For **main**, keep the AI step advisory:

```
- name: AI production triage
  if: failure()
  run: |
    echo "Generate summary, likely cause, risk, rollback plan"
    echo "Require manual approval before production changes"
```